Sprint 2: Values, Data Types, Variables, and Type Checks (ිංහල පරිවර්තනය

0. What Sprint 2 Is About (Sprint 2 එකෙන් අපි ඉගෙනගන්නේ මොනවාද?)

- **Concept Explanation (Sinhala):** Sprint 2 එකේදී අපි ප්‍රධාන වශයෙන්ම ඉගෙනගන්නේ JavaScript කොහොමද data store කරගන්නේ (තැන්පත් කරගන්නේ) සහ ඒවා තේරුම් ගන්නේ කොහොමද කියන එකයි.

  Sprint 1 එකේදී ඔයාලා ඉගෙනගත්තේ button click එකක් වගේ action එකකට JavaScript පාවිච්චි කරලා web page එකක් respond කරවන්නේ (ප්‍රතිචාර දක්වන්නේ) කොහොමද කියලා. Sprint 2 එකේදී අපි ඉගෙනගන්නේ මේ වගේ actions සිද්ධ වෙනකොට JavaScript වැඩ කරන්නේ මොන වගේ data එක්කද කියලා.

  උදාහරණ විදිහට:

  - Button එකක් click කරද්දී: මොන value එකද වෙනස් වුණේ?
  - Form එකක් submit කරද්දී: user type කරපු text එක මොකක්ද?
  - Score එක update වෙද්දී: ඒ score එක ඇත්තටම number එකක්ද?
  - Task එකක් add කරද්දී: ඒ task එක store වෙලා තියෙන්නේ කොහේද?

  Sprint 2 එකේ ප්‍රධානම ඉලක්කය තමයි: **Data ආරක්ෂිතව store කරගන්න එක සහ ඔයාලා වැඩ කරන්නේ මොන වගේ value එකක් එක්කද කියලා හරියටම තේරුම් ගන්න එක.**
- **Code Block:** *(මෙම කොටසට අදාළව code block එකක් නොමැත)*
- **Code Breakdown (Sinhala):** *(නොමැත)*

1. The Core Idea of Sprint 2 (Sprint 2 එකේ මූලිකම අදහස)

- **Concept Explanation (Sinhala):** JavaScript programs වැඩ කරන්නේ values (දත්ත කෑබලි) එක්කයි. `value` එකක් කියන්නේ JavaScript වලට වැඩ කරන්න පුළුවන් දත්ත කෑබැල්ලකටයි. හැම value එකකටම අනන්‍ය වෙච්ච `type` (වර්ගයක්) එකක් තියෙනවා.

  JavaScript program එකකින් සාමාන්‍යයෙන් values එක්ක මේ දේවල් 3 කරනවා:

  1. Values store කරනවා (තැන්පත් කරනවා).
  2. Values read කරනවා (කියවනවා).
  3. Values change කරනවා (වෙනස් කරනවා).

  Value එකක් store කරගන්න අපි සාමාන්‍යයෙන් පාවිච්චි කරන්නේ `variable` එකක්.
- **Code Block:**

```
let age = 25;
```

- **Code Breakdown (Sinhala):**
  - `let age` : `age` නමින් variable එකක් හදනවා.
  - `= 25` : ඒ හදපු variable එක ඇතුළට `25` කියන value එක store කරනවා.

2. What Is a Value? (Value එකක් කියන්නේ මොකක්ද?)

- **Concept Explanation (Sinhala):** `Value` එකක් කියන්නේ JavaScript වලට පාවිච්චි කරන්න පුළුවන් සැබෑ දත්ත (actual data) වලටයි. අපිට මේ values `console.log()` පාවිච්චි කරලා screen එකේ print කරගන්න පුළුවන්.
- **Code Block:**

```
console.log(100);
console.log("JavaScript");
console.log(true);
```

- **Code Breakdown (Sinhala):**
  - `console.log(100)` : `100` කියන number value එක console එකේ print කරයි.
  - `console.log("JavaScript")` : `"JavaScript"` කියන string (text) value එක print කරයි.
  - `console.log(true)` : `true` කියන boolean value එක print කරයි.

3. What Is a Data Type? (Data Type එකක් කියන්නේ මොකක්ද?)

- **Concept Explanation (Sinhala):** `Data type` එකකින් කරන්නේ තමන් ළඟ තියෙන value එක මොන වර්ගයේ එකක්ද කියලා JavaScript වලට කියන එකයි. සරලවම කිව්වොත් value එකක් අයිති වෙන category එක (වර්ගය) තමයි data type එකක් කියන්නේ.

  Data types වැදගත් වෙන්නේ ඇයි? අපිට ලැබෙන value එකේ type එක අනුව JavaScript හැසිරෙන විදිහ වෙනස් වෙනවා. පහත කේතය බලන්න:
- **Code Block:**

```
console.log(5 + 10);     // Output: 15
console.log("5" + 10);   // Output: "510"
```

- **Code Breakdown (Sinhala):**
  - පළමු පේළියේ `5` කියන්නේ **Number** එකක්. ඒ නිසා JavaScript සාමාන්‍ය එකතු කිරීමක් කරලා `15` ලබා දෙනවා.
  - දෙවැනි පේළියේ `"5"` කියන්නේ **String** (text) එකක්. ඒ නිසා JavaScript කරන්නේ ඒ text එක පිටුපසින් `10` අලවන එකයි (concatenation). ප්‍රතිඵලය වෙන්නේ `"510"`.

4. The Main Data Types in Sprint 2 (Sprint 2 වලදී හමුවන ප්‍රධාන Data Types)

- **Concept Explanation (Sinhala):** JavaScript වල තියෙන ප්‍රධාන data types වර්ග කිහිපයක් අපි හඳුනාගන්න ඕනේ:
  1. **Number:** ඉලක්කම්/සංඛ්‍යා සඳහා.
  2. **String:** Text (අකුරු/වචන) සඳහා.
  3. **Boolean:** `true` හෝ `false` සඳහා.
  4. **undefined:** Variable එකක් හැදුවත් තවම value එකක් දීලා නැති අවස්ථා සඳහා.
  5. **null:** හිතාමතාම හිස් (intentionally empty) අගයක් දැමීම සඳහා.
  6. **Object:** එකිනෙකට සම්බන්ධ දත්ත එකට එකතු කර තැබීම සඳහා.
  7. **Symbol:** සුවිශේෂී, වෙනස් නොවන අනන්‍යතාවයක් (unique identifier) සඳහා.
  8. **BigInt:** ඉතා විශාල පූර්ණ සංඛ්‍යා (integers) සඳහා.
- **Code Block:** *(මෙම කොටසට අදාළව code block එකක් නොමැත)*
- **Code Breakdown (Sinhala):** *(නොමැත)*

5. Number (සංඛ්‍යා)

- **Concept Explanation (Sinhala):** ගණිතමය වැඩකටයුතු සහ සංඛ්‍යා ලේඛන තබා ගැනීමට `Number` පාවිච්චි කරනවා. වයස, ලකුණු, මිල ගණන්, උෂ්ණත්වය වගේ දේවල් වලට අපි යොදාගන්නේ Numbers.
- **Code Block:**

```
let score = 10;
let bonus = 5;
console.log(score + bonus); // Output: 15

let price = 19.99;
let quantity = 3;
let total = price * quantity;
console.log(total); // Output: 59.97
```

- **Code Breakdown (Sinhala):**
  - `score` සහ `bonus` කියන variables දෙකටම numbers එකතු කරලා, ඒවා එකට එකතු කරලා screen එකේ පෙන්වනවා.
  - දශම සංඛ්‍යා (decimals) සහ සෘණ සංඛ්‍යා (negative numbers) වුණත් JavaScript සලකන්නේ `Number` data type එකක් විදිහටමයි.

6. String (අකුරු / වචන / Text)

- **Concept Explanation (Sinhala):** `String` එකක් කියන්නේ text එකකටයි. Strings ලියද්දී හැමතිස්සෙම quotes (උද්ධෘත ලකුණු) ඇතුළේ ලියන්න ඕනේ. අපිට single quotes (`'`), double quotes (`"`) හෝ backticks (`` ` ``) පාවිච්චි කරන්න පුළුවන්.

  වැදගත් මතක තබා ගත යුතු කරුණක්: `25` කියන්නේ Number එකක්. නමුත් `"25"` කියන්නේ String එකක්. මේ දෙක පෙනුමෙන් සමාන වුණාට වැඩ කරන්නේ ක්‍රම දෙකකටයි!
- **Code Block:**

```
let name = "Ada";
let course = 'JavaScript';
let message = `Welcome`;

console.log(name);
console.log(course);
console.log(message);
```

- **Code Breakdown (Sinhala):**
  - මෙතැනදී `name` , `course` , `message` කියන variables තුනටම strings ආදේශ කරලා තියෙනවා. ඒවා console එකේ පිළිවෙළින් "Ada", "JavaScript", "Welcome" ලෙස පෙන්වනවා.

7. Boolean (සත්‍ය / අසත්‍ය)

- **Concept Explanation (Sinhala):** `Boolean` අගයක් විය හැක්කේ දෙකෙන් එකක් පමණි: එක්කෝ `true` (සත්‍ය/ඔව්) නැතහොත් `false` (අසත්‍ය/නැහැ). මෙය yes/no හෝ on/off තත්ත්වයන් පාලනය කිරීමට යොදාගනී (උදා: user කෙනෙක් log වෙලාද නැද්ද, වැඩක් කරලා ඉවරද නැද්ද වගේ දේවල්).
- **Code Block:**

```
let isLoggedIn = true;
let isAdmin = false;
let taskCompleted = false;

console.log(taskCompleted); // Output: false
```

- **Code Breakdown (Sinhala):**
  - `isLoggedIn` variable එකට `true` දාලා තියෙන්නේ දැනට user log වෙලා ඉන්නවා කියන අදහසින්.
  - `taskCompleted` එකට `false` දාලා තියෙන නිසා තවම task එක ඉවර නැහැ කියන එක සරලවම තේරුම් ගන්න පුළුවන්.

8. undefined (නිර්වචනය නොකළ)

- **Concept Explanation (Sinhala):** `undefined` කියන්නේ variable එකක් create (declare) කරලා තිබුණත්, ඒකට තවම කිසිම value එකක් දාලා නැති අවස්ථාවටයි. ඒ කියන්නේ JavaScript දැනට ඒ variable එක ඇතුළේ කිසිම value එකක් දකින්නේ නැහැ.
- **Code Block:**

```
let age;
console.log(age); // Output: undefined

age = 25;
console.log(age); // Output: 25
```

- **Code Breakdown (Sinhala):**
  - `let age;` පේළියේදී variable එක හැදුවා විතරයි, අගයක් දුන්නේ නැහැ. ඒ නිසා පළමු `console.log(age)` එකෙන් `undefined` කියලා පෙන්වනවා.
  - පසුව `age = 25;` මඟින් අගයක් ලබා දුන් පසු දෙවැනි `console.log(age)` එකෙන් `25` මුද්‍රණය කරනවා.

9. null (හිස් අගය)

- **Concept Explanation (Sinhala):** `null` කියන්නේ හිතාමතාම "කිසිවක් නැහැ" (intentionally empty) කියලා පෙන්වන්න programmer විසින්ම දාන value එකක්.

  undefined සහ null අතර වෙනස:

  - `undefined` කියන්නේ තවම කිසිම value එකක් දීලා නැහැ (නොදන්නා අගයක්).
  - `null` කියන්නේ දැනුවත්වම, හිතාමතාම හිස් අගයක් ලබා දී තිබෙනවා (හිස් බව දන්නවා).
- **Code Block:**

```
let firstName;
let middleName = null;

console.log(firstName);  // Output: undefined
console.log(middleName); // Output: null
```

- **Code Breakdown (Sinhala):**
  - `firstName` වලට value එකක් නොදුන් නිසා එය `undefined` වේ.
  - `middleName` වලට හිතාමතාම `null` ලබා දී ඇති බැවින් එය `null` ලෙස පෙන්වයි.

10. Object (වස්තුව / දත්ත එකතුවක්)

- **Concept Explanation (Sinhala):** `Object` එකක් පාවිච්චි කරන්නේ එකිනෙකට සම්බන්ධ දත්ත කිහිපයක් එකට එකතු කරලා තියාගන්නයි. Objects වල තියෙන්නේ key-value pairs (යතුර සහ අගය යුගල). අපිට object එකක තියෙන දත්ත කියවන්න dot notation (තිත පාවිච්චි කිරීමේ ක්‍රමය) යොදාගන්න පුළුවන්.
- **Code Block:**

```
const player = {
  username: "coder123",
  score: 50,
  online: true
};

console.log(player.username); // Output: coder123
console.log(player.score);    // Output: 50
```

- **Code Breakdown (Sinhala):**
  - මෙතැනදී `player` කියන object එක ඇතුළේ `username` , `score` , `online` කියලා properties 3ක් එකතු කරලා තියෙනවා.
  - `player.username` කියලා ලියද්දී player object එකේ තියෙන username එක වෙන `"coder123"` කියවන්න පුළුවන්. වෙන වෙනම variables 3ක් හදනවාට වඩා මේ ක්‍රමය හරිම පහසුයි.

11. Symbol & 12. BigInt (Symbol සහ BigInt පිළිබඳ සරල හැඳින්වීම)

- **Concept Explanation (Sinhala):** * **Symbol:** හැමතිස්සෙම එකිනෙකට වෙනස් අද්විතීය (unique) අගයක් නිර්මාණය කිරීමට පාවිච්චි කරයි. එකම නමින් symbols දෙකක් හැදුවත් ඒවා එකිනෙකට සමාන වෙන්නේ නැහැ.
  - **BigInt:** JavaScript වල සාමාන්‍ය Number එකකින් ලියන්න පුළුවන් ප්‍රමාණයට වඩා ඉතා විශාල පූර්ණ සංඛ්‍යා ලිවීමට පාවිච්චි කරයි. BigInt අගයක් අගට `n` අකුර එකතු කරන්න ඕනේ (උදා: `100n`).

  (සටහන: ආධුනිකයෙක් විදිහට මේ data types දෙක ගැන මූලික අවබෝධයක් තිබීම දැනට ප්‍රමාණවත්ය.)
- **Code Block:**

```
const firstId = Symbol("id");
const secondId = Symbol("id");
console.log(firstId === secondId); // Output: false

const bigNumber = 1234567890123456789012345678900n;
console.log(typeof bigNumber); // Output: bigint
```

- **Code Breakdown (Sinhala):**
  - `firstId === secondId` පරීක්ෂා කිරීමේදී `false` ලැබෙන්නේ හැම Symbol එකක්ම එකිනෙකට වෙනස් නිසාය.
  - `bigNumber` අගට `n` ඇති නිසා එය `bigint` ලෙස හඳුනා ගනී.

13. What Is a Variable? (Variable එකක් කියන්නේ මොකක්ද?)

- **Concept Explanation (Sinhala):** `Variable` එකක් කියන්නේ යම්කිසි value එකක් තාවකාලිකව දමා තැබීමට පාවිච්චි කරන නමකටයි (container / ලේබලයක්). Variables නැතිව අපිට එකම අගය නැවත නැවත ලියන්න වෙනවා. Variables පාවිච්චි කරලා අපිට දත්ත store කරන්න, ආයේ පාවිච්චි කරන්න, සහ update (වෙනස්) කරන්න ලේසියෙන්ම පුළුවන්.
- **Code Block:**

```
let score = 10;
let bonus = 5;
console.log(score + bonus); // Output: 15
```

- **Code Breakdown (Sinhala):**
  - `let score = 10` මඟින් `10` කියන value එක `score` කියන නම යටතේ store කරනවා. පසුව අපිට අවශ්‍ය තැන්වලදී මේ නම පාවිච්චි කරලා ගණනය කිරීම් කරන්න පුළුවන්.

14. Parts of a Variable Statement (Variable ප්‍රකාශනයක කොටස්)

- **Concept Explanation (Sinhala):** JavaScript වල variable එකක් ලියනකොට ඒ ප්‍රකාශනයේ ප්‍රධාන කොටස් කිහිපයක් අඩංගු වෙනවා.
  - `let` / `const` : Keyword එක (variable එක හදන්න ගන්නා විශේෂ වචනය).
  - `score` / `username` : Variable Name එක (අපි දෙන නම).
  - `=` : Assignment Operator එක (දකුණු පැත්තේ තියෙන දේ වම් පැත්තේ තියෙන variable එකට දාන ලකුණ).
  - `10` / `"Ada"` : Value එක (අගය).
  - `;` : End of statement (පේළිය අවසන් කිරීමේ ලකුණ).
- **Code Block:**

```
let score = 10;
const username = "Ada";
```

- **Code Breakdown (Sinhala):**
  - පළමු පේළියෙන් කියන්නේ: "score නමින් variable එකක් සාදා එහි අගය 10 කරන්න."
  - දෙවැනි පේළියෙන් කියන්නේ: "username නමින් වෙනස් නොවන variable එකක් සාදා එහි අගය 'Ada' කරන්න."

15. Declaration & 16. Initialization & 17. Assignment (ප්‍රකාශ කිරීම, මුලින්ම අගයක් දීම සහ අගයක් ආදේශ කිරීම)

- **Concept Explanation (Sinhala):** මේ සංකල්ප තුන පටලවා නොගෙන තේරුම් ගැනීම ඉතා වැදගත්:
  - **Declaration:** Variable එකක නම පමණක් නිර්මාණය කිරීම (තවම value එකක් දී නැත, එනිසා එහි අගය `undefined` වේ).
  - **Initialization:** Variable එකකට මුලින්ම (පළමු වතාවට) value එකක් ලබා දීම.
  - **Assignment:** `=` ලකුණ මඟින් දකුණු පැත්තේ ඇති value එක වම් පැත්තේ ඇති variable එක ඇතුළට දැමීම. (මතක තබා ගන්න: JavaScript වලදී `=` ලකුණෙන් අදහස් වෙන්නේ "සමානයි" කියන එක නෙවෙයි, "දකුණේ තියෙන දේ වමේ තියෙන එකට දාන්න" කියන එකයි).
- **Code Block:**

```
let age;          // Declaration (දැනට undefined වේ)
age = 25;         // Initialization (පළමු වරට value එකක් ලැබුණි)

let score = 10;   // Declaration සහ Initialization පේළියකෙම සිදු කර ඇත
```

- **Code Breakdown (Sinhala):**
  - `let age;` මඟින් memory එකේ `age` නමින් ඉඩක් වෙන් කරනවා පමණි.
  - `age = 25;` මඟින් එම ඉඩට `25` කියන අගය ඇතුළු කරයි.

18. Reassignment (නැවත අගයක් ලබා දීම)

- **Concept Explanation (Sinhala):** `Reassignment` කියන්නේ කලින් value එකක් තිබුණු variable එකකට, ඊට පස්සේ අලුත් value එකක් දෙන එකටයි. මෙතැනදී පැරණි value එක මැකී ගොස් අලුත් value එක එහි තැන්පත් වේ.
- **Code Block:**

```
let score = 10;
score = 15;
console.log(score); // Output: 15
```

- **Code Breakdown (Sinhala):**
  - `score` මුලින් `10` ලෙස initialize කර ඇත.
  - පසුව `score = 15` ලෙස reassigned කර ඇත. දැන් එහි පැරණි අගය (10) නැත, ඇත්තේ `15` පමණි.

19. let vs 20. const (let සහ const අතර වෙනස)

- **Concept Explanation (Sinhala):** * **let:** Variable එකේ තියෙන value එක පස්සේ කාලෙක වෙනස් කරන්න (reassign කරන්න) අවශ්‍ය වෙනවා නම් අපි පාවිච්චි කරන්නේ `let` . (උදා: game score එකක්, item count එකක්).
  - **const:** Variable එකට දාන value එක කවදාවත් වෙනස් කරන්න ඉඩ දෙන්නේ නැතිනම් (reassign කරන්න බැරි වෙන්න lock කරන්න ඕනේ නම්) අපි පාවිච්චි කරන්නේ `const` . (උදා: උපන් වර්ෂය, app එකේ නම). `const` variable එකකට අලුතින් value එකක් දෙන්න හැදුවොත් JavaScript වලින් error එකක් පෙන්වනවා.
- **Code Block:**

```
let score = 10;
score = 15; // Allowed (අවසර ඇත)

const birthYear = 2008;
// birthYear = 2009; // Error! (reassign කිරීමට නොහැක)
```

- **Code Breakdown (Sinhala):**
  - `score` යනු `let` variable එකක් නිසා එය සාර්ථකව `15` දක්වා වෙනස් කළ හැක.
  - `birthYear` යනු `const` variable එකක් නිසා එය වෙනස් කිරීමට උත්සාහ කළහොත් program එක crash වේ (වැඩ කිරීම නවතී).

21. Beginner Rule: Use const First (ආධුනිකයන් සඳහා රීතිය: මුලින්ම const පාවිච්චි කරන්න)

- **Concept Explanation (Sinhala):** ආධුනිකයෙක් විදිහට කේත ලියද්දී ලේසිම ක්‍රමයක් තමයි: හැමතිස්සෙම මුලින්ම `const` පාවිච්චි කරන්න. ඔයාට ඒ variable එකේ අගය පස්සේ වෙනස් කරන්නම වෙනවා කියලා විශ්වාස නම් විතරක් ඒක ලියද්දී `let` පාවිච්චි කරන්න. මේ පුරුද්ද නිසා ඔයා අතින් වැරදීමකින් variables වල අගයන් වෙනස් වීම වැළකෙනවා (code එක වඩාත් ආරක්ෂිත වෙනවා).
- **Code Block:**

```
const username = "Ada"; // වෙනස් නොවේ
let userScore = 0;      // වෙනස් වීමට ඉඩ ඇත
userScore = userScore + 10;
```

- **Code Breakdown (Sinhala):**
  - `username` කවදාවත් වෙනස් වෙන්නේ නැති නිසා එය `const` ලෙසත්, `userScore` නිතර වෙනස් වන නිසා එය `let` ලෙසත් ආරක්ෂිතව නිර්මාණය කර ඇත.

22. Debug Challenge: const Reassignment Error

- **Concept Explanation (Sinhala):** `const` මඟින් හදපු variable එකකට නැවත අගයක් දීමට යාමෙන් සිදුවන වැරැද්ද නිවැරදි කරගන්නා ආකාරය පහත පරිදි වේ:
- **Code Block:**

```
// Broken Code (වැරදි ක්‍රමය):
// const points = 5;
// points = 8; // TypeError: Assignment to constant variable.

// Fixed Code (නිවැරදි ක්‍රමය):
let points = 5;
points = 8;
console.log(points); // Output: 8
```

- **Code Breakdown (Sinhala):**
  - වැරදි ක්‍රමයේදී `points` යනු `const` එකක් නිසා එය වෙනස් කිරීමට ඉඩ නොදේ.
  - නිවැරදි ක්‍රමයේදී `points` යන්න `let` ලෙස වෙනස් කළ නිසා සාර්ථකව `8` ලෙස වෙනස් වී ඇත.

23. const with Objects (const සහ Objects අතර ඇති සුවිශේෂී සම්බන්ධය)

- **Concept Explanation (Sinhala):** මෙය ඉතාමත් වැදගත් කරුණක්! `const` **object** එකක් ඇතුළේ තියෙන **properties** අපිට ඕන වෙලාවක වෙනස් කරන්න පුළුවන්. `const` වලින් කරන්නේ ඒ variable එක වෙනත් අලුත් object එකකට point කරන එක (පිටතින් වෙනස් කරන එක) වැළැක්වීම පමණයි. නමුත් object එක ඇතුළට ගිහින් එහි ඇති දත්ත (properties) වෙනස් කිරීමට බාධාවක් නැත.

  සරලවම: `const` මඟින් *variable* එකේ නම වෙනත් තැනකට යොමු කිරීම වළක්වයි, නමුත් *object* එක ඇතුළත *"freeze"* කරන්නේ නැත.
- **Code Block:**

```
const player = { name: "Ada", active: true };

player.active = false; // Allowed! (අවසර ඇත - ඇතුළත දත්ත වෙනස් කළ හැක)
console.log(player);   // Output: { name: "Ada", active: false }

// player = { name: "Grace" }; // Error! (තහනම් - සම්පූර්ණයෙන්ම වෙනස්
object එකක් ආදේශ කළ නොහැක)
```

- **Code Breakdown (Sinhala):**
  - `player.active = false` මඟින් පවතින object එකේම දත්ත වෙනස් කළ නිසා එය සාර්ථක වේ.
  - `player = { name: "Grace" }` ලියූ විට, එය මුළු variable එකම අලුත් වස්තුවකට (object එකකට) හැරවීමට උත්සාහ කරන නිසා JavaScript මඟින් error එකක් ලබා දේ.

24. Why Variable Names Matter & Case Sensitivity & camelCase (Variables නම් කිරීමේ නීති)

- **Concept Explanation (Sinhala):** * **විස්තරාත්මක නම් යෙදීම:** Variables වලට `x` , `y` , `z` වගේ තේරුමක් නැති නම් දාන්නේ නැතිව, අදාළ value එක පැහැදිලි වෙන විදිහේ නමක් දාන්න (උදා: `totalPrice` , `userAge`).
  - **Case Sensitivity (කැපිටල්/සිම්පල් වෙනස):** JavaScript වලදී variable එකක අකුරු ලියන ආකාරය ඉතා වැදගත්. `score` සහ `Score` කියන්නේ variables දෙකක්!
  - **camelCase රටාව:** වචන දෙකක් හෝ කිහිපයක් එකතු කර variable එකක් ලියද්දී, පළමු වචනය සිම්පල් අකුරින් පටන්ගෙන, ඊළඟට එන හැම වචනයකම පළමු අකුර කැපිටල් අකුරින් ලිවීමේ ක්‍රමයයි (උදා: `totalPrice` , `isLoggedIn`).
- **Code Block:**

```
let score = 10;
let Score = 20;
console.log(score); // Output: 10
console.log(Score); // Output: 20

let totalItemPrice = 150; // camelCase රටාව
```

- **Code Breakdown (Sinhala):**
  - `score` සහ `Score` යනු අකුරු වල විශාලත්වය වෙනස් නිසා සම්පූර්ණයෙන්ම වෙනස් variables දෙකක් ලෙස JavaScript සලකයි.
  - `totalItemPrice` යන්න කියවීමට පහසු වන සේ camelCase මඟින් ලියා ඇත.

25. Boolean Naming Pattern (Boolean නම් කිරීමේ පොදු රටාව)

- **Concept Explanation (Sinhala):** Boolean variables (true/false) නම් කරද්දී ඒවා සාමාන්‍යයෙන් ප්‍රශ්නයක් වගේ හැඟෙන වචන වලින් පටන් ගන්නවා.
  - `is` (උදා: `isLoggedIn` , `isActive`)
  - `has` (උදා: `hasAccount` , `hasFinished`)
  - `can` (උදා: `canEdit` , `canDelete`)
  - `should` (උදා: `shouldSave`)

  මේ පුරුද්ද නිසා code එක කියවන ඕනෑම කෙනෙකුට මේ variable එක ඇතුළේ තියෙන්නේ `true` හෝ `false` අගයක් කියලා එකපාරටම තේරෙනවා.
- **Code Block:**

```
let isLoggedIn = true; // "User ලොග් වෙලාද? ඔව් (true)"
let canEdit = false;    // "Edit කරන්න පුළුවන්ද? නැහැ (false)"
```

- **Code Breakdown (Sinhala):**
  - `isLoggedIn` සහ `canEdit` කියන නම් දුටු පමණින් ඒවා boolean variables බව හඳුනාගත හැක.

26. Dynamic Typing (වෙනස් වන සුළු data types)

- **Concept Explanation (Sinhala):** JavaScript කියන්නේ `dynamically typed` භාෂාවක්. ඒ කියන්නේ එකම variable එකකට එක වෙලාවක Number එකක් දාලා, ඊට පස්සේ වෙලාවක ඒ variable එකටම String එකක් දාන්න පුළුවන්. Variable එකේ type එක ඕනෑම වෙලාවක වෙනස් වෙන්න පුළුවන්.

  මෙයින් සිදුවන වැරදි (Concatenation Bug): මේ නම්‍යශීලී බව නිසා සමහර වෙලාවට ලොකු bugs (වැරදි) ඇතිවෙන්න පුළුවන්. උදාහරණයක් විදිහට:
- **Code Block:**

```
let score = 10;
score = "10"; // Number එකක්ව තිබූ score දැන් String එකක් වී ඇත!
console.log(score + 5); // Output: "105" (නමුත් අපට අවශ්‍ය වූයේ 15 ලබා
ගැනීමටයි!)
```

- **Code Breakdown (Sinhala):**
  - `score` මුලින් Number (10) එකක් වුවද, පසුව එය String ("10") එකක් බවට පත් කරන ලදී.
  - `+ 5` එකතු කිරීමේදී JavaScript කරන්නේ එකතු කිරීම වෙනුවට text දෙකක් එකට අලවන එකයි. එම නිසා අපට බලාපොරොත්තු නොවූ "105" අගය ලැබේ.
  - ආරක්ෂිත පුරුද්ද: *variable එකක් එක data type එකකින් තවත් data type එකකට නිකරුණේ වෙනස් කරන්න එපා.*

27. What Is typeof? (typeof operator එක)

- **Concept Explanation (Sinhala):** `typeof` කියන්නේ තමන් ළඟ තියෙන value එකක් මොන data type එකටද අයිති වෙන්නේ කියලා JavaScript වලින් අහලා දැනගන්න පුළුවන් විශේෂ විධානයක් (operator එකක්).
- **Code Block:**

```
console.log(typeof 25);          // Output: "number"
console.log(typeof "hello");     // Output: "string"
console.log(typeof true);        // Output: "boolean"
console.log(typeof undefined);   // Output: "undefined"
```

- **Code Breakdown (Sinhala):**
  - `typeof` මඟින් අදාළ අගයන්ගේ දත්ත වර්ගය හඳුනාගෙන ඒවායේ නම් string එකක් විදිහට ආපසු ලබා දෙයි.

28. The typeof null Bug (typeof null හි ඇති ඓතිහාසික වැරැද්ද)

- **Concept Explanation (Sinhala):** මෙය JavaScript ඉගෙන ගන්න හැමෝම දැනගත යුතුම විශේෂ කරුණක්. JavaScript නිර්මාණය කරපු මුල් කාලයේ සිදුවූ වැරැද්දක් (bug එකක්) නිසා `typeof null` මුද්‍රණය කළ විට ලැබෙන්නේ `"object"` කියාය.

  නමුත් ඇත්තටම `null` කියන්නේ object එකක් නෙවෙයි, එය වෙනමම ප්‍රාථමික data type එකක්. නමුත් අදටත් මේ වැරැද්ද වෙනස් කර නැත (එසේ කළහොත් පැරණි වෙබ් අඩවි බිඳ වැටිය හැකි බැවිනි). එබැවින් `typeof null` යනු `"object"` බව නිතරම මතක තබා ගන්න.
- **Code Block:**

```
let selectedUser = null;
console.log(typeof selectedUser); // Output: "object"
```

- **Code Breakdown (Sinhala):**
  - `selectedUser` හි අගය `null` වුවත්, `typeof` පරීක්ෂා කිරීමේදී ප්‍රතිඵලය `"object"` ලෙස ලැබේ.

29. Using typeof for Debugging (typeof operator එක debugging සඳහා යොදාගැනීම)

- **Concept Explanation (Sinhala):** අපේ program එකේ ගණන් හිලව් හෝ වෙනත් දේවල් බලාපොරොත්තු නොවන විදිහට හැසිරෙන කොට, අදාළ variables වල ඇත්තටම තියෙන්නේ මොන වගේ values ද කියලා බලාගන්න `typeof` අපිට ගොඩක් උදව් වෙනවා.
- **Code Block:**

```
let userAge = "25";
console.log(typeof userAge); // Output: "string"
```

- **Code Breakdown (Sinhala):**
  - බාලු බැල්මට `userAge` යනු `25` (සංඛ්‍යාවක්) ලෙස පෙනුණද, `typeof` මඟින් එය ඇත්තටම string (text) එකක් බව අපට ඔප්පු කර පෙන්වයි. දැන් අපිට එය නිවැරදි කරගත හැක.

30. Practice: typeReport Function (සරල typeReport ශ්‍රිතය)

- **Concept Explanation (Sinhala):** ඕනෑම value එකක් දුන් විට එහි type එක වාක්‍යයක් ලෙස ආපසු දෙන සරල function එකක් පහත පරිදි නිර්මාණය කළ හැක.
- **Code Block:**

```
function typeReport(value) {
  return "The value is a " + typeof value;
}

console.log(typeReport("hello")); // Output: "The value is a string"
console.log(typeReport(42));      // Output: "The value is a number"
console.log(typeReport(null));    // Output: "The value is a object"
(typeof null bug එක නිසා)
```

- **Code Breakdown (Sinhala):**
  - `typeReport` function එකට ලැබෙන `value` එකේ type එක `typeof` මඟින් සොයාගෙන, එය `"The value is a "` යන string කොටස සමඟ සම්බන්ධ කර (concatenate කර) ආපසු ලබා දේ.

31. Primitive vs Non-Primitive Values (ප්‍රාථමික සහ ප්‍රාථමික නොවන අගයන්)

- **Concept Explanation (Sinhala):** JavaScript වල ඇති values ප්‍රධාන කොටස් 2කට බෙදන්න පුළුවන්:
  1. **Primitive Values:** සරල තනි අගයන් වේ (number, string, boolean, undefined, null, symbol, bigint).
  2. **Non-Primitive Values:** සංකීර්ණ දත්ත ව්‍යුහයන් වේ (objects, arrays, functions).

  මේ වර්ග දෙක memory එකේ වැඩ කරන විදිහ සහ copy වෙන විදිහ සම්පූර්ණයෙන්ම වෙනස්.
- **Code Block:** *(මෙම කොටසට අදාළව code block එකක් නොමැත)*
- **Code Breakdown (Sinhala):** *(නොමැත)*

32. Primitive Values Are Copied by Value (Primitive values පිටපත් වෙන්නේ අගය සෘජුවම ලබා දීමෙනි)

- **Concept Explanation (Sinhala):** `Primitive values` එකක් තවත් variable එකකට copy කරනකොට සිද්ධ වෙන්නේ ඒ value එකේ අලුත්ම, ස්වාධීන (independent) copy එකක් හැදෙන එකයි.

  ඒ නිසා එක variable එකක් පස්සේ වෙනස් කළත්, අනිත් variable එකේ තියෙන අගය වෙනස් වෙන්නේ නැහැ. ඒවා එකිනෙකට ස්වාධීනයි.
- **Code Block:**

```
let a = 10;
let b = a; // a හි ඇති 10 හි ස්වාධීන පිටපතක් b ට ලැබේ
a = 20;

console.log(a); // Output: 20
console.log(b); // Output: 10
```

- **Code Breakdown (Sinhala):**
  - `b = a` ලියන විට `b` ට ලැබුණේ `10` හි සම්පූර්ණ වෙනම අගයකි.
  - පසුව `a = 20` ලෙස වෙනස් කළද, `b` සතු අගය තවදුරටත් `10` ලෙසම සුරැකී පවතී.

33. Objects Are Copied by Reference (Objects පිටපත් වෙන්නේ යොමුව / Pointer එක ලබා දීමෙනි)

- **Concept Explanation (Sinhala):** නමුත් `Objects` (සහ Arrays) ක්‍රියා කරන්නේ මීට වඩා සම්පූර්ණයෙන්ම වෙනස් ක්‍රමයකටයි. Object එකක් තවත් variable එකකට copy කරද්දී අලුත් object එකක් හැදෙන්නේ නැහැ.

  සිද්ධ වෙන්නේ ඒ මුල් object එක memory එකේ තියෙන තැනට යන ලිපිනය **(reference / pointer එක)** අලුත් variable එකට ලැබෙන එකයි. ඒ කියන්නේ variables දෙකම දැන් පොයින්ට් කරන්නේ (යොමු වෙන්නේ) memory එකේ තියෙන එකම එක object එකකටයි!

  එක් variable එකක් හරහා object එක ඇතුළේ දත්ත වෙනස් කළහොත්, අනෙක් variable එකෙන් බලද්දීත් ඒ වෙනස එලෙසම දකින්න ලැබෙනවා.
- **Code Block:**

```
const user = { name: "Alice" };
const sameUser = user; // user සතු reference එකම sameUser ට ලැබේ

user.name = "Ada"; // original object එක වෙනස් කරයි
console.log(sameUser.name); // Output: "Ada"
```

- **Code Breakdown (Sinhala):**
  - `sameUser` යනු වෙනම සාදාගත් අලුත් object එකක් නොවේ. එයද `user` සමඟ එක්ව එකම object එක දෙස බලා සිටී.
  - `user.name = "Ada"` ලෙස වෙනස් කළ විට, variables දෙකම බලන්නේ එකම දත්තය දෙස බැවින් `sameUser.name` පරීක්ෂා කළ විටද `"Ada"` ලෙස ප්‍රතිඵලය ලැබේ.

34. Core Sprint 2 Code Pattern (මූලික කේත රටාව සහ එහි පැහැදිලි කිරීම)

- **Concept Explanation (Sinhala):** Sprint 2 හි අප ඉගෙනගත් සියලුම මූලික කරුණු එකට එකතු කළ කේත රටාවක් පහතින් දැක්වේ. මෙය හොඳින් අධ්‍යයනය කරන්න.
- **Code Block:**

```
let age;
console.log(age); // Output: undefined

age = 25;
console.log(age); // Output: 25

const user = { name: "Alice" };
const sameUser = user;
user.name = "Ada";
console.log(sameUser.name); // Output: Ada
```

- **Code Breakdown (Sinhala):**
  1. `let age;` මඟින් variable එකක් සාදයි, අගයක් නැති නිසා එය `undefined` වේ.
  2. `age = 25;` මඟින් එයට ප්‍රථම වරට අගයක් (initialize) ලබා දෙයි.
  3. `const user` මඟින් object එකක් සාදා එහි reference එක `user` තුළ තබයි.
  4. `const sameUser = user` මඟින් එම reference එකම `sameUser` වෙත පිටපත් කරයි (දෙකම එකම ලිපිනයට යොමු වේ).
  5. `user.name = "Ada"` මඟින් පොදු object එකේ ඇති නම වෙනස් කරයි.
  6. `sameUser.name` මුද්‍රණය කළ විට `"Ada"` ලෙස ප්‍රතිඵලය ලැබේ, මන්ද දෙදෙනාම එකම object එක බෙදාගන්නා බැවිනි.

35. Sprint 2 Starter Practice (ආරම්භක ප්‍රායෝගික අභ්‍යාසය)

- **Concept Explanation (Sinhala):** ඔබේ පරිගණකයේ ක්‍රියාත්මක කර බැලීම සඳහා සරල ආරම්භක කේතයක් මෙන්න.
- **Code Block:**

```
let score = 10;
const player = {
  name: "Ada",
  active: true
};

score = 15;
player.active = false;

console.log(score);         // Output: 15
console.log(player);        // Output: { name: "Ada", active: false }
console.log(typeof null);   // Output: object
```

- **Code Breakdown (Sinhala):**
  - `score` යනු `let` variable එකක් නිසා අගය සාර්ථකව වෙනස් වේ.
  - `player` යනු `const` object එකක් වුවද එහි ඇතුළත ඇති `active` property එක වෙනස් කිරීමට අවසර ඇත.
  - `typeof null` මඟින් ඓතිහාසික bug එකක් වන `"object"` ප්‍රතිඵලය පෙන්වයි.

36. Common Sprint 2 Mistakes (නිතරම සිදුවන වැරදි සහ ඒවා වළක්වා ගන්නා ආකාරය)

- **Concept Explanation (Sinhala):** ආධුනිකයන් නිතරම කරන වැරදි කිහිපයක් සහ ඒවා නිවැරදි කරගන්නා ආකාරය:
  1. **Reassigning const (const වෙනස් කිරීමට යාම):**
     - වැරදි ක්‍රමය: `const score = 10; score = 20;` (Error එකක් ලැබේ).
     - නිවැරදි ක්‍රමය: අගය වෙනස් කිරීමට අවශ්‍ය නම් `let` පාවිච්චි කරන්න. (`let score = 10; score = 20;`).
  2. **Thinking const Freezes Objects (const මඟින් object එකක් freeze කරනවා යැයි සිතීම):**
     - නිවැරදි අවබෝධය: `const` මඟින් කරන්නේ variable එක වෙනස් කිරීම වැළැක්වීම පමණි. Object එකේ ඇතුළත දත්ත වෙනස් කිරීමට හැකිය.
  3. **Confusing undefined and null (undefined සහ null පටලවා ගැනීම):**
     - *undefined*: තවම කිසිම value එකක් දීලා නැහැ.
     - *null*: හිතාමතාම හිස් අගයක් ලබා දී ඇත.
  4. **Thinking Object Copy Creates a Separate Object (Object එකක් copy කළ විට අලුත්ම object එකක් හැදෙනවා යැයි සිතීම):**
     - නිවැරදි අවබෝධය: Object එකක් copy කළ විට ලැබෙන්නේ reference (ලිපිනය) එකක් පමණි. Variables දෙකම බලන්නේ එකම object එක දෙසය.
- **Code Block:** *(මෙම කොටසට අදාළව code block එකක් නොමැත)*
- **Code Breakdown (Sinhala):** *(නොමැත)*

37. Practice Tasks & Debug Challenges (ප්‍රායෝගික අභ්‍යාස සහ ගැටලු විසඳීම)

- **Concept Explanation (Sinhala):** ඔබේ දැනුම පරීක්ෂා කරගැනීම සඳහා පහත අභ්‍යාසයන් සහ Debug Challenges තනිවම විසඳීමට උත්සාහ කරන්න.
- **Code Block:**

```
// Warm-Up Drill 1: let variables 3ක් සාදා ඒවා නැවත වෙනස් (reassign)
කරන්න.
let score = 10;
let level = 1;
let message = "Start";
score = 20;
level = 2;
message = "Continue";

// Warm-Up Drill 2: const variables 3ක් සාදා ඒවා මුද්‍රණය කරන්න.
const appName = "Study Tracker";
const maxScore = 100;
const isPublic = true;

// Debug Challenge 1: පහත කේතයේ ඇති වැරැද්ද කුමක්ද?
// const points = 5;
// points = 8;
// පිළිතුර: const variables reassign කළ නොහැක. 'let points = 5;' ලෙස
වෙනස් කළ යුතුය.

// Debug Challenge 2: පහත කේතයේ output එක කුමක් විය හැකිද?
const user = { name: "Ada" };
const sameUser = user;
sameUser.name = "Grace";
console.log(user.name);
// පිළිතුර: "Grace" (මන්දියත් user සහ sameUser යොමුවන්නේ එකම object එකට
බැවිනි).
```

- **Code Breakdown (Sinhala):**
  - `user` සහ `sameUser` එකම reference එකක් බෙදාගන්නා නිසා, `sameUser.name` වෙනස් කළ විට `user.name` ද වෙනස් වේ.

38. Sprint 2 Mental Models (මනසේ මවාගත යුතු නිවැරදි රූප / රටා)

- **Concept Explanation (Sinhala):** JavaScript variables සහ values ක්‍රියා කරන ආකාරය පහසුවෙන් මතක තබා ගැනීමට මෙම ක්‍රම භාවිත කරන්න:
  1. **Variable as a Label (ලේබලයක් ලෙස සිතන්න):** Primitive value එකක් සඳහා variable එකක් කියන්නේ අගය අලවා ඇති ලේබලයක් වැනිය. `let age = 25` කියන්නේ `age` ලේබලය `25` මත අලවා ඇත. `age = 26` කළ විට එම ලේබලය ගලවා `26` මත අලවයි.
  2. **Object Reference (නූලකින් බැඳ ඇති බැලූනයක් ලෙස සිතන්න):** Object එකක් යනු බැලූනයක් බඳුය. Variable එක යනු එම බැලූනය බැඳ ඇති නූලයි. `const sameUser = user` ලියන විට, සිදුවන්නේ එම බැලූනයටම තවත් නූලක් (තවත් variable එකක්) ගැටගැසීම වැනි වැඩකි. නූල් දෙකෙන්ම අල්ලාගෙන සිටින්නේ එකම බැලූනයයි.
  3. **let vs const (නූල වෙනස් කළ හැකිද නැද්ද යන්න):**
     - `let score` : මේ නූල ලිහිල් කර වෙනත් ඕනෑම බැලූනයක ගැටගැසිය හැක.
     - `const score` : මේ නූල කවදාවත් ගලවා වෙනත් බැලූනයක ගැටගැසිය නොහැක (එය ස්ථිරවම බැඳ ඇත). නමුත් බැලූනය ඇතුළත ඇති වාතය හෝ හැඩය (properties) අපට වෙනස් කළ හැක.
- **Code Block:** *(මෙම කොටසට අදාළව code block එකක් නොමැත)*
- **Code Breakdown (Sinhala):** *(නොමැත)*

39. Key Vocabulary (ප්‍රධාන වචන මාලාව)

- **Concept Explanation (Sinhala):** * **Value:** JavaScript වලට වැඩ කළ හැකි ඕනෑම දත්ත කෑබැල්ලක්.
  - **Data Type:** යම් කිසි value එකක් අයිති වන වර්ගය (category එක).
  - **Variable:** Value එකක් තැන්පත් කරගැනීමට දෙනු ලබන නම.
  - **Declaration:** Variable එකක් මුලින්ම නිර්මාණය කිරීම.
  - **Initialization:** Variable එකකට ප්‍රථම වරට අගයක් ලබා දීම.
  - **Assignment:** `=` ලකුණ මඟින් අගයක් variable එකක් තුළට දැමීම.
  - **Reassignment:** දැනටමත් අගයක් ඇති variable එකකට අලුත් අගයක් ලබා දීම.
  - **let:** නැවත වෙනස් කළ හැකි variables සෑදීමට ගන්නා keyword එක.
  - **const:** නැවත වෙනස් කළ නොහැකි (reassign කළ නොහැකි) variables සෑදීමට ගන්නා keyword එක.
  - **Dynamic Typing:** Variable එකකට ඕනෑම වෙලාවක ඕනෑම data type එකක value එකක් දැමීමට JavaScript ඉඩ දීමේ ස්වභාවය.
  - **typeof:** Value එකක data type එක කුමක්දැයි පරීක්ෂා කර බැලීමට ගන්නා operator එක.
  - **Primitive Value:** වෙන වෙනම ස්වාධීනව copy වන සරල දත්ත වර්ග (numbers, strings, booleans, undefined, null, etc.).
  - **Reference:** Object එකක් පවතින memory ලිපිනයට ඇති සම්බන්ධතාවය (pointer එක).
- **Code Block:** *(මෙම කොටසට අදාළව code block එකක් නොමැත)*
- **Code Breakdown (Sinhala):** *(නොමැත)*

40. Revision Questions & Answers (පුනරීක්ෂණ ප්‍රශ්න සහ පිළිතුරු)

- **Concept Explanation (Sinhala):** විභාගයට හෝ සම්මුඛ පරීක්ෂණයකට පෙර මෙම ප්‍රශ්න සහ පිළිතුරු කියවා ඔබේ දැනුම තහවුරු කරගන්න.
  1. **ප්‍රශ්නය: Value එකක් යනු කුමක්ද?**
     - පිළිතුර: JavaScript වලට ක්‍රියාත්මක කළ හැකි ඕනෑම දත්ත කෑබැල්ලකි. (උදා: `25` , `"hello"` , `true`).
  2. **ප්‍රශ්නය: Data type එකක් යනු කුමක්ද?**
     - පිළිතුර: යම් value එකක් අයත් වන දත්ත වර්ගයයි (උදා: number, string, boolean).
  3. **ප්‍රශ්නය: Variable එකක් යනු කුමක්ද?**
     - පිළිතුර: දත්තයක් (value එකක්) තබා ගැනීමට අප භාවිත කරන නමකි.
  4. **ප්‍රශ්නය: Declaration සහ Initialization අතර වෙනස කුමක්ද?**
     - පිළිතුර: Declaration මඟින් variable එකේ නම පමණක් සාදයි. Initialization මඟින් එයට ප්‍රථම වතාවට අගයක් ලබා දෙයි.
  5. **ප්‍රශ්නය: Reassignment යනු කුමක්ද?**
     - පිළිතුර: පවතින variable එකකට පසුව වෙනත් අලුත් අගයක් ලබා දීමයි.
  6. **ප්‍රශ්නය: `let` භාවිත කළ යුත්තේ කවදාද?**
     - පිළිතුර: Variable එකේ ඇති අගය ඉදිරියේදී වෙනස් කිරීමට අවශ්‍ය වන විටදීය.
  7. **ප්‍රශ්නය: `const` භාවිත කළ යුත්තේ කවදාද?**
     - පිළිතුර: Variable එකේ ඇති අගය ඉදිරියේදී වෙනස් කිරීමට ඉඩ නොදෙන අවස්ථාවලදීය (reassign කිරීම වැළැක්වීමට).
  8. **ප්‍රශ්නය: `const` variable එකක් reassign කළ හැකිද?**
     - පිළිතුර: නැත, එය කළ නොහැක (Error එකක් ලැබෙනු ඇත).
  9. **ප්‍රශ්නය: `const` object එකක ඇතුළත ඇති දත්ත (properties) වෙනස් කළ හැකිද?**
     - පිළිතුර: ඔව්, පුළුවන. `const` මඟින් වළක්වන්නේ මුළු object එකම වෙනස් කිරීම පමණි. ඇතුළත දත්ත වෙනස් කිරීමට අවසර ඇත.
  10. **ප්‍රශ්නය: `undefined` සහ `null` අතර ඇති ප්‍රධාන වෙනස කුමක්ද?**
      - පිළිතුර: `undefined` යනු variable එකක් සාදා ඇතත් තවම අගයක් ලැබී නැති අවස්ථාවයි. `null` යනු හිතාමතාම එහි කිසිවක් නැති බව පෙන්වීමට යොදන හිස් අගයකි.
  11. **ප්‍රශ්නය: `typeof null` මඟින් ලැබෙන ප්‍රතිඵලය කුමක්ද? එය පටලැවිලි සහගත වන්නේ ඇයි?**
      - පිළිතුර: එය `"object"` ලෙස ලැබේ. මෙය JavaScript හි පවතින පැරණි ඓතිහාසික වැරැද්දක් (bug) නිසා සිදුවන්නකි. ඇත්තටම `null` යනු object එකක් නොවේ.
  12. **ප්‍රශ්නය: Dynamic typing යනු කුමක්ද?**
      - පිළිතුර: එකම variable එකකට විවිධ අවස්ථාවලදී විවිධ data types වලට අයත් අගයන් (උදා: මුලින් number එකක්, පසුව string එකක්) ලබා දීමට JavaScript ඉඩ දීමයි.
  13. **ප්‍රශ්නය: Dynamic typing නිසා ඇති විය හැකි bug එකක් පැහැදිලි කරන්න.**
      - පිළිතුර: Number එකක් තිබූ variable එකක් වැරදීමකින් string එකක් බවට පත් වූ විට, එකතු කිරීම් වලදී සංඛ්‍යා එකතු වෙනවා වෙනුවට ඒවා එක ළඟ ඇලවීම (string concatenation) සිදුවේ (උදා: `"10" + 5 = "105"`).
  14. **ප්‍රශ්නය: Primitive value එකක් copy වෙන්නේ කෙසේද?**
      - පිළිතුර: ඒවා copy වන්නේ **copied by value** ක්‍රමයටයි. එනම් එකිනෙකට සම්පූර්ණයෙන්ම ස්වාධීන වූ වෙනම පිටපතක් සාදා දීමෙනි.
  15. **ප්‍රශ්නය: Object එකක් copy වෙන්නේ කෙසේද?**
      - පිළිතුර: ඒවා copy වන්නේ **copied by reference** ක්‍රමයටයි. එනම් අලුත් පිටපතක් සාදන්නේ නැතිව, පවතින object එකේම ලිපිනය (reference එක) ලබා දීමෙනි. variables දෙකම එකම object එකක් බෙදා ගනී.
- **Code Block:** *(මෙම කොටසට අදාළව code block එකක් නොමැත)*
- **Code Breakdown (Sinhala):** *(නොමැත)*

Minimum Passing Standard (අවම සමත් වීමේ මට්ටම)

මෙම පාඩමෙන් ඔබ ලබාගත යුතු වැදගත්ම අවබෝධය සාරාංශයක් ලෙස මෙන්න:

- **Primitive Value:** වෙන වෙනම ස්වාධීනව පිටපත් වේ.
- **Object Value:** එකම ලිපිනය (reference) බෙදා ගනී.

```
// Primitive Example:
let x = 10;
let y = x;
x = 20;
console.log(y); // Output: 10 (ස්වාධීන පිටපතකි)

// Object Example:
const obj1 = { name: "Ada" };
const obj2 = obj1;
obj2.name = "Grace";
console.log(obj1.name); // Output: "Grace" (පොදු ලිපිනයකි)
```